

## Notes de cours

Ces notes de cours son en partie inspirées du cours de Cyril Gavaille [2].

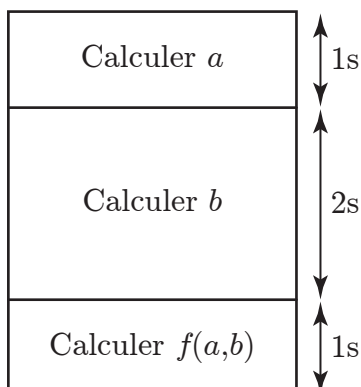
## I Introduction

- Une machine *séquentielle* effectue les opérations les unes après les autres,
- Une machine *parallèle* peut effectuer plusieurs opérations simultanément,
  - Il s'agit de décomposer un calcul en plusieurs sous-calculs indépendants,
- Un *système distribué* est composé de plusieurs entités de calcul :
  - Possibilité de beaucoup plus d'entités de calcul qu'une machine parallèle,
  - Distance physique potentiellement importante entre deux de ces entités.

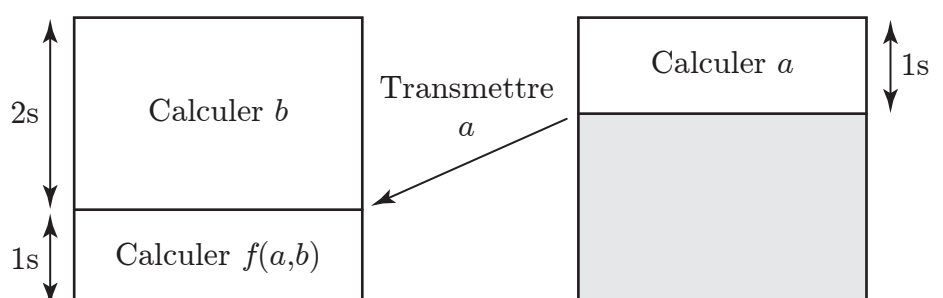
**Exemple.** On veut calculer une certaine quantité  $f(a, b)$  (par exemple  $a + b$ ,  $a \times b$ , ...). Pour calculer  $a$ , il faut 1 seconde, pour calculer  $b$ , il faut 2 secondes et pour calculer  $f(a, b)$  à partir de  $a$  et de  $b$ , il faut 1 seconde.

- Sur une machine séquentielle, il faudra donc 4 secondes pour calculer  $f(a, b)$ ,
- Sur un système distribué avec deux processeurs, il faudrait 2 secondes pour calculer  $b$  sur l'un des processeurs. Pendant ce temps, l'autre processeur peut calculer  $a$ . Une fois que  $a$  et  $b$  sont calculés, un des deux processeurs peut envoyer son résultat ( $a$  ou  $b$ ) à l'autre qui peut alors calculer  $f(a, b)$ . Il faut donc 3 secondes pour calculer  $f(a, b)$  au lieu de 4 secondes.

Machine sequentielle



Système distribue avec deux processeurs



L'objectif de ce cours est l'étude d'algorithmes distribués et la mise en oeuvre de ces algorithmes. Nous utiliserons la librairie MPI pour la mise en oeuvre.

## I.1 Architectures

Dans l'exemple ci-dessus, dans le cas du système distribué, on ne compte pas le temps qu'il faut pour que le  $b$  soit envoyé du premier processeur au second (ou pour que  $a$  soit envoyé du second processeur au premier). Si les processeurs de ce système sont (très) distants, le temps nécessaire à cet envoi peut être significativement plus grand que le temps de calcul ! Ceci pose la question de l'*architecture* du système distribué.

**Mémoire partagée.** Toutes les entités de calcul stockent les données dans une mémoire commune :

- Cas d'une machine parallèle,
- Communications peu coûteuses.

**Mémoire distribuée.** Chaque entité de calcul stocke ses données dans une mémoire qui n'est accessible qu'à elle-même :

- Cas d'un réseau d'ordinateurs (calcul en grille, Seti@home, Folding@home...),
- Communications coûteuses (et temps de calcul négligeable). Les entités de calcul communiquent en s'envoyant des messages (contenant par exemple le résultat de leur calcul).

**Exemple (suite).** Reprenons l'exemple précédent, dans le cas du système distribué et supposons en outre que, pour envoyer un message de l'un des processeurs à l'autre, il faut 1 seconde.

- Si le premier processeur (celui qui calcule  $b$ ) est celui qui calcule  $f(a, b)$ , alors il faut toujours 3 secondes pour calculer  $f(a, b)$ , puisque le calcul de  $a$  ne prend qu'une seconde,
- Si le deuxième processeur (celui qui calcule  $a$ ) est celui qui calcule  $f(a, b)$ , alors il faudra 4 secondes pour calculer  $f(a, b)$  !

**Exemple.** Considérons deux systèmes distribués constitué de 3 entités de calcul numérotées I, II, et III :

- Dans le premier, chaque entité de calcul est reliée à chacune des deux autres. Chaque entité de calcul peut donc communiquer avec les deux autres direc-

tement,

- Dans le deuxième, I est relié à II, et II est relié à III. Si I et III veulent communiquer, il faut donc envoyer un message à II, qui doit ensuite le transmettre en envoyant un autre message.

Ces deux systèmes sont très différents, même si le nombre d'entités de calcul est le même. Le graphe sous-jacent est donc une des caractéristiques importantes d'un système distribué.

## I.2 Modèle de communication par passage de messages

Ce modèle de système distribué, très répandu, est celui qui sera abordé dans ce cours (et qui est implémenté par la librairie MPI).

- Toutes les entités de calcul exécutent le **même** programme,
  - Avantage : il n'y a qu'un seul programme à écrire (heureusement car le nombre d'entités de calcul est potentiellement grand!).
  - Inconvénient : il faut que les entités de calcul commencent par décider (collectivement) de l'attribution des différents calculs (qui calcule  $a$  et qui calcule  $b$  dans notre premier exemple...) Ceci n'est pas une tâche simple car toutes les entités de calcul exécutent le même programme et donc, sans autre information, vont faire la même chose. Il faudra donc que le système distribué commence par décider, de manière collective la tâche de chaque entité (exemple de l'élection ou du consensus).
- Chaque entité de calcul
  - peut communiquer avec ses voisins dans le graphe du système distribué (envoyer des messages à ses voisins, recevoir des messages de ses voisins),
  - ne connaît pas l'état des autres entités de calcul, à moins d'avoir reçu un message qui décrit cet état, et là encore, peut-être que ce message ne donne pas des informations à jour (le message a peut-être été envoyé avant que les informations n'aient été modifiées!)
- Lorsqu'une entité de calcul attend un message, elle est bloquée (ne peut pas faire de calcul) jusqu'à réception du message (en général, parce qu'elle a besoin de l'information contenue dans ce message pour poursuivre le calcul).

Reprenons notre premier exemple. Naïvement :

| Processeur 1                 | Processeur 2                |
|------------------------------|-----------------------------|
| Calculer $b$                 | Calculer $a$                |
| Recevoir $a$ du processeur 2 | Envoyer $a$ au processeur 1 |
| Calculer $f(a, b)$           |                             |

**Question.** Il ne s'agit pas du même programme! Comment faire pour que les deux processeurs exécutent le même programme?

**Réponse.** Utiliser l'identifiant (unique) de chaque processeur (ci-dessus, 1 et 2).

| Processeur 1                            | Processeur 2                            |
|-----------------------------------------|-----------------------------------------|
| <b>Si</b> identifiant==1 <b>alors</b> , | <b>Si</b> identifiant==1 <b>alors</b> , |
| Calculer $b$                            | Calculer $b$                            |
| Recevoir $a$ du processeur 2            | Recevoir $a$ du processeur 2            |
| Calculer $f(a, b)$                      | Calculer $f(a, b)$                      |
| <b>Sinon</b> ,                          | <b>Sinon</b> ,                          |
| Calculer $a$                            | Calculer $a$                            |
| Envoyer $a$ au processeur 1             | Envoyer $a$ au processeur 1             |

Cet exemple est bien-sûr très simplifié. En pratique, il est possible (et courant) qu'une des entités de calcul soit éteinte (ou en panne), par exemple dans le cas du calcul en grille. Dans une telle situation, la distribution des tâches sur les différentes entités de calcul doit être fait plus subtilement!

Une autre question importante se pose lors de la mise en oeuvre d'algorithmes distribués. En effet, une entité de calcul ne connaît le système que localement : elle ne connaît que l'identifiant de ses voisins directs. Elle ne connaît la valeur d'aucune des variables stockée par les autres entités de calcul.

**Question.** Si une variable est mise à jour sur une entité de calcul, comment transmettre cette mise-à-jour aux autres entités de calcul (qui ne sont pas nécessairement voisines de celle-ci)?

La réponse pratique à cette question est loin d'être évidente : par quels liens doit-on faire passer l'information? Comment informer les entités de calcul intermédiaires qu'elles doivent transmettre le message et à qui? Comment gérer les redondances (le même message reçu plusieurs fois par la même entité de la part de plusieurs autres entités différentes)? D'autre part, une réponse à cette question n'est possible que si chaque entité de calcul est connectée à toutes les autres par

au moins un chemin ! Tester cette condition est déjà une tâche très complexe ! Pour pouvoir espérer répondre à cette question, il faut examiner l'organisation des connections entre les différentes entités de calcul, ce qui nécessite d'étudier la notion suivante :

**Graphe d'un système distribué.** Dans un système distribué, deux entités de calcul ne sont pas forcément voisines. Pour représenter un système distribué, on utilise un graphe dont les sommets sont les entités de calcul et dont les arêtes relient deux entités de calcul voisines.

**Exemple.** Considérons un système distribué fait de quatre entités de calcul  $P_1, \dots, P_4$  telles que chaque entité de calcul est voisine de toutes les autres, sauf  $P_1$  qui n'est pas voisine de  $P_4$ . Quel est le graphe de ce système ? Pour chaque paire d'entité de calcul, combien faut-il de messages pour transmettre une information de l'une à l'autre des entités de calcul ?

### I.3 Rappels de théorie des graphes

Soit  $G$  un graphe dont l'ensemble des sommets est  $V$ , et dont l'ensemble des arêtes est  $E$ . On note  $G = (V, E)$ ,  $V = V(G)$  et  $E = E(G)$ .

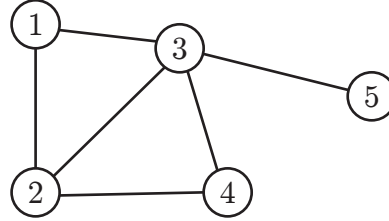
- On note souvent  $n$  le nombre de sommets de  $G$  et  $m$  le nombre d'arêtes de  $G$ ,
- Le degré d'un sommet  $u \in V$  dans  $G$ , noté  $\deg_G(u)$ , est le nombre de ses voisins dans  $G$  (le nombre de sommet avec lesquels il partage une arête),
- Un chemin dans  $G$  est une suite de sommets  $u_0, \dots, u_k$  telle que deux sommets consécutifs ( $u_{i-1}$  et  $u_i$ ) sont reliés par une arête de  $G$ . La longueur d'un tel chemin est  $k$ ,
- Le graphe  $G$  est connexe si, pour toute paire de sommets  $u$  et  $v$  de  $G$ , il existe un chemin dans  $G$  entre  $u$  et  $v$ ,
- Lorsqu'il existe au moins un chemin dans  $G$  entre deux sommets  $u$  et  $v$ , on appelle distance de  $u$  et de  $v$ , et on note  $d(u, v)$  la longueur d'un chemin le plus court entre  $u$  et  $v$ ,
- Si  $G$  est connexe, le diamètre de  $G$  est  $\text{diam}(G) = \max_{u, v \in V} d(u, v)$ ,
- Si  $G$  est connexe, l'eccentricité d'un sommet  $u$  dans  $G$ , notée  $\text{ecc}_G(u)$  est sa

distance la plus grande à un autre sommet de  $G$ , c'est-à-dire

$$\text{ecc}_G(u) = \max_{v \in V} d(u, v).$$

L'eccentricité de  $G$ , notée  $\text{ecc}(G)$  est la valeur minimale de  $\text{ecc}_G(u)$ . Un centre de  $G$  est un sommet  $u$  de  $G$  tel que  $\text{ecc}_G(u) = \text{ecc}(G)$ .

**Exemple :**



Ce graphe est tel que  $n = 5$  et  $m = 6$ . Il est connexe. Son diamètre est 2 et son eccentricité est 1. Le sommet 3 est son unique centre.

**Proposition :**  $\sum_{u \in V} \deg_G(u) = 2m.$

*Preuve.* Le membre de droite de cette inégalité compte le nombre d'incidences entre un sommet et une arête. Puisqu'une arête est incidente à deux sommets, ce nombre est égal au double du nombre d'arêtes.  $\square$

**Proposition :**  $\frac{1}{2} \text{diam}(G) \leq \text{ecc}_G(u) \leq \text{diam}(G)$  pour tout  $u \in V(G)$ .

*Preuve.* L'eccentricité de  $u$  dans  $G$  est la distance entre  $u$  et un certain autre sommet de  $G$ . Puisque le diamètre est la plus grande distance entre deux sommets du graphe, nous obtenons immédiatement l'inégalité de droite.

Considérons deux sommets  $v$  et  $w$  de  $G$ . D'après l'inégalité triangulaire,

$$d(v, w) \leq d(u, v) + d(u, w).$$

Puisque les distances  $d(u, v)$  et  $d(u, w)$  sont toutes deux majorées par  $\text{ecc}_G(u)$ , nous pouvons en déduire :

$$d(v, w) \leq 2\text{ecc}_G(u).$$

Cette majoration est vraie pour toute paire de sommets  $v$  et  $w$ . Elle reste donc vraie pour le diamètre de  $G$  :

$$\text{diam}(G) \leq 2\text{ecc}_G(u).$$

Il suffit alors de diviser cette majoration par 2 pour obtenir l'inégalité de gauche

dans l'énoncé du théorème.

□

## II Complexité et modèles

Il existe plusieurs mesures de la rapidité des algorithmes distribués. Nous en présenterons quelques unes dans ce chapitre. Ces mesures sont faites sous certaines hypothèses, rassemblées dans des modèles que nous examinerons ensuite.

### II.1 Quelques mesures de complexité

Dans le cas d'une machine séquentielle, la complexité d'un algorithme est mesurée par le nombre d'instructions qui doivent être exécutées.

Dans le cas d'un système distribué, la notion de complexité est plus subtile :

- Plusieurs tâches peuvent être effectuées en même temps,
- Lorsque les entités de calcul sont distantes, l'acheminement des messages est significativement plus lent que les calculs. Ainsi, le temps d'exécution d'un programme distribué est essentiellement dû à la transmission des messages.

Pour ces raisons, plusieurs complexités différentes peuvent être utilisées :

**Définition (complexité en temps asynchrone).** La complexité en temps d'un algorithme asynchrone distribué  $A$  sur un système distribué de graphe  $G$ , notée  $\text{TEMPS}(A, G)$  est le nombre d'unités de temps nécessaire à l'exécution de  $A$  sur  $G$  dans le pire des cas, *chaque message prenant au plus une unité de temps pour transiter le long d'une arête de  $G$ .*

Une autre définition de la complexité s'intéresse au nombre de messages qui sont transmis lors de l'exécution de l'algorithme :

**Définition (complexité en nombre de messages).** La complexité en nombre de messages d'un algorithme distribué  $A$  sur un système distribué de graphe  $G$ , notée  $\text{MESSAGE}(A, G)$  est le nombre de messages échangés durant l'exécution de  $A$  sur  $G$  dans le pire des cas.

La complexité en temps est mesurée dans le cas d'algorithmes *asynchrones*, c'est-à-dire que l'algorithme progresse indépendamment sur les différentes entités de calculs, sauf lorsque l'une d'entre elle attends un message d'une autre entité de calcul. Il existe des algorithmes *synchrones*, pour lesquels, chaque entité de calcul

effectue une suite de cycles, de façon que le  $i$ -ème cycle est effectué au même moment par toutes les entités de calcul. Lors d'un cycle, un algorithme synchrone se déroule de la manière suivante :

- 1) Effectuer des calculs,
- 2) Envoyer des messages à au moins une des entités de calcul voisines,
- 3) Recevoir des messages d'au moins une des entités de calcul voisines,

Attention, les calculs effectués pendant un cycle sur une entité de calcul doivent être obligatoirement terminés avant l'envoi et la réception des messages pendant ce même cycle et sur la même entité de calcul. Plus précisément, il n'est pas possible d'utiliser les données recues pendant ce cycle pour faire ces calculs. Par exemple, le pseudo-code suivant correspond à plus d'un cycle, même si les deux envois sont fait vers des entités de calcul différentes :

**Envoyer**  $x$  à l'entité  $A$

**Recevoir**  $y$  à l'entité  $B$

**Envoyer**  $x + y$  à l'entité  $C$

Pendant un même cycle, puisque chaque envoi de messages est suivi de réception de messages, deux entités de calcul doivent chacune attendre que l'autre ait envoyé son message pour pouvoir passer au cycle suivant, ce qui synchronise le système distribué. La complexité en temps des algorithmes synchrones peut être définie de manière simplifiée :

**Definition (complexité en temps synchrone).** La complexité en temps d'un algorithme distribué synchrone  $A$  sur un système distribué de graphe  $G$ , notée  $\text{TEMPS}(A, G)$  est le nombre de cycles nécessaire à l'exécution de  $A$  sur  $G$  dans le pire des cas.

## II.2 modèles

Les mesures de complexité que nous venons de définir ne sont réalistes que sous certaines hypothèses que nous allons maintenant énoncer. Les trois premières hypothèses sont les suivantes :

- Pas d'erreurs : les processeurs ne sont jamais en panne, et les liens entre eux sont toujours utilisables. Lorsqu'un message est envoyé le long d'une arête du graphe d'un système distribué, ce message arrive nécessairement à l'entité



de calcul qui est au bout de l'arête. Si  $\{a, b\}$  est une telle arête, un message peut aller de  $a$  vers  $b$  au même moment qu'un autre message va de  $b$  vers  $a$  (pas de collision). Enfin, les messages envoyés le long d'une arête dans un sens arrivent dans leur ordre d'émission.

- Temps de calcul négligeable : le temps de calcul sur chaque entité de calcul est négligeable (considéré comme nul) par rapport au temps de transit des messages.
- Identité des entités de calcul : chaque entité de calcul a un identifiant unique (qui n'est l'identifiant d'aucune autre entité de calcul dans le système distribué). Cet identifiant est en général un nombre.

Ces trois premières hypothèses peuvent être complétées de différentes manières. Selon les hypothèses qu'on ajoute, on obtient un *modèle* qui permet d'examiner différents aspects d'un algorithme distribué :

**Modèle LOCAL.** Ce modèle permet d'examiner le comportement local d'un algorithme distribué en négligeant l'influence de la taille des messages transmis.

- Mode synchrone.
- Pas de limite sur la taille des messages.

**Modèle CONGEST.** Ce modèle permet d'examiner l'influence de la taille des messages transmis sur le comportement d'un algorithme distribué.

- Mode synchrone.
- Tailles des messages limitées à  $O(\log(n))$  bits, où  $n$  est le nombre d'entités de calcul.

**Modèle ASYNC.** Ce modèle permet d'examiner le comportement d'un algorithme distribué en mode asynchrone.

- Mode asynchrone.

### III Quelques problèmes distribués

Dans un système distribué, certains problèmes classiques doivent être résolus, afin de permettre les calculs (comme par exemple décider de l'attribution des tâches). Parmi ces problèmes, on trouve :

- Le problème de l'élection : les entités de calcul doivent choisir (élire) l'une d'entre elles. Cette entité de calcul élue aura un rôle particulier dans le calcul,

- Le problème du consensus : chaque entité de calcul dispose d'une valeur initiale. Les entités de calcul doivent toutes adopter une et une seule de ces valeurs,
- Le problème de la diffusion : un message, initialement connu par une des entités de calcul, doit être transmis à toutes les entités de calcul.
- Le problème du calcul d'un arbre couvrant...

### III.1 Le problème de la diffusion

Diffuser un message consiste à transmettre un message  $M$  d'une entité de calcul  $r$  à toutes les autres entités de calcul. Le message peut être transmis directement de  $r$  à une autre entité de calcul, ou bien transiter par différentes entités de calcul (dans ce cas, les entités de calcul autres que  $r$  doivent recevoir le message et le retransmettre à certains de leur voisins).

**Proposition.** Un algorithme de diffusion  $A$  sur un système distribué de graphe  $G$  avec  $n$  sommets vérifie :

- MESSAGE( $A, G$ )  $\geq n - 1$ ,
- TEMPS( $A, G$ )  $\geq \text{ecc}(r)$ ,

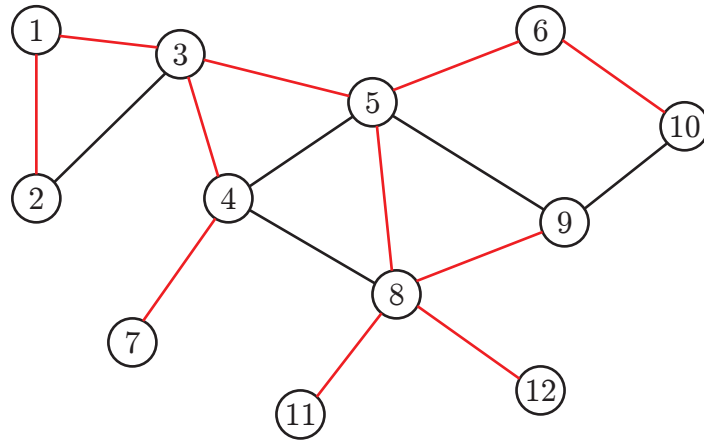
*Preuve.* Une transmission ne délivre le message qu'à une seule entité de calcul. Puisqu'il y a  $n - 1$  entités de calcul à informer, il faut donc au moins  $n - 1$  transmissions du message.

En outre, il y a une entité de calcul  $f$  à distance  $\text{ecc}_G(r)$ . Puisque la transmission d'un message d'une entité à une entité voisine prend un temps unitaire, il faudra au moins un temps de  $\text{ecc}_G(r)$  pour que le message soit transmis de  $r$  à  $f$ .  $\square$

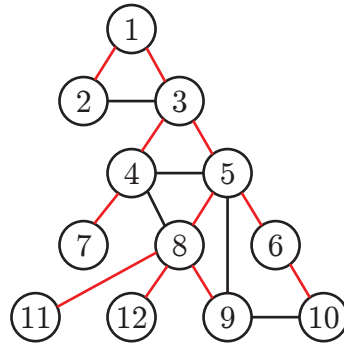
#### III.1.1 Arbre de diffusion

Considérons un système distribué de graphe  $G$  connexe, avec  $n$  sommets et  $m$  arêtes. Il n'est pas nécessaire d'utiliser toutes les arêtes de  $G$  pour diffuser un message à partir d'une des entités de calcul.

**Exemple.** Dans le graphe ci-dessous, pour transmettre un message  $M$  du sommet 1 à tous les autres sommets, il suffit d'utiliser les arêtes rouges.



Le sous-graphe fait des arêtes rouges s'appelle un *arbre couvrant* : ce sous-graphe est connexe et ne contient pas de cycles (donc c'est un arbre). En outre, tous les sommets du graphe sont des sommets de l'arbre (donc il est couvrant). Si on choisit un sommet  $r$  qui sera la racine de l'arbre (il s'agit, naturellement, de l'entité qui est la source du message), nous pouvons représenter l'arbre de manière que tous les sommets à une distance donnée soient à la même *hauteur* :



La hauteur de l'arbre  $T$  fait des arêtes rouges, notée  $\text{HAUTEUR}(T)$  est l'écart maximal de ses sommets, ici 4. Chaque sommet est relié, vers le haut, à son unique père et, vers le bas, à un ou plusieurs fils (le sommet 1 a deux fils, le sommet 6 en a un seul, et le sommet 8 a trois fils).

On note  $\text{PÈRE}(u)$  le père du sommet  $u$  et  $\text{FILS}(u)$  l'ensemble des fils de  $u$ . Si, chaque entité de calcul d'un système distribué connaît son père et la liste de ses fils, l'algorithme suivant permet de résoudre le problème de diffusion en utilisant  $n - 1$  messages. Cet algorithme s'appelle  $\text{CAST}_T(r)$  :

**Si** identifiant  $\neq r$  **alors**,

Recevoir  $M$  de  $\text{PÈRE}(\text{identifiant})$ ,

**Fin si**,

**Pour tout**  $u \in \text{FILS}(\text{identifiant})$  **faire**,

Envoyer  $M$  à  $u$ ,

**Fin pour.**

Cet algorithme envoie autant de messages que d'arêtes dans l'arbre  $T$ . D'après la proposition suivante, le nombre de messages transmis est donc bien  $n - 1$  :

**Proposition.** Un arbre  $T$  avec  $n$  sommets admet  $n - 1$  arêtes.

*Preuve.* La preuve se déroule par induction sur  $n$ . Si  $n = 1$ , l'arbre ne contient pas d'arêtes, donc son nombre d'arêtes est  $0 = n - 1$ .

Supposons que pour un certain nombre entier  $n > 1$ , le nombre d'arêtes d'un arbre avec  $n - 1$  sommets soit  $n - 2$ . Considérons un arbre  $T$  avec  $n$  sommets, et l'une de ses feuilles  $f$ . Soit  $T'$  l'arbre qu'on obtient en enlevant  $f$  et l'arête qui relie cette feuille à son père. Ce nouvel arbre admet  $n - 1$  sommets. D'après l'hypothèse de récurrence, il a donc  $n - 2$  arêtes. Puisque  $T$  a une arête de plus que  $T'$ , il a nécessairement  $n - 1$  arêtes.  $\square$

**Proposition.**

- i.  $MESSAGE(CAST_T(r), G) = n - 1$ ,
- ii.  $TEMPS(CAST_T(r), G) = hauteur(T)$ .

### III.1.2 Inondation

Lorsqu'on ne dispose pas d'un arbre recouvrant, la technique de l'inondation, noté FLOOD( $r$ ) peut être utilisée :

**Si** identifiant =  $r$  **alors**,

Pour tout voisin  $v$ , Envoyer  $M$  à  $v$ ,

**Sinon**,

Recevoir  $M$  d'un voisin  $v$ ,

Envoyer  $M$  à tous voisin  $u$  différent de  $v$ .

**Fin si**

La complexité de l'algorithme d'inondation est donnée par la proposition suivante :

**Proposition.**

- i.  $MESSAGE(FLOOD(r), G) = 2m - n + 1$ ,
- ii.  $TEMPS(FLOOD(r), G) = ecc_G(r)$ .

*Preuve.* Le nombre de messages envoyés par le sommet  $r$  est  $\deg(r)$  et le nombre

de messages envoyés par tout autre sommet  $u$  est  $\deg(u) - 1$ . Par conséquent,

$$\text{MESSAGE}(\text{FLOOD}(r), G) = \left[ \sum_{u \in V} \deg(u) \right] - n + 1 = 2m - n + 1.$$

En outre, après un temps  $t$ , les sommets de  $G$  qui ont reçu le message sont ceux qui sont à distance au plus  $t$  de  $r$ . Pour cette raison, la complexité en temps  $\text{TEMPS}(\text{FLOOD}(r), G)$  est égale à  $\text{ecc}_G(r)$ .  $\square$

### III.1.3 Tas binaire dans un graphe complet

Lorsque le graphe  $G$  du système distribué est complet (c'est-à-dire que chaque entité de calcul est voisine de toutes les autres), que les entités de calcul sont numérotées de 0 à  $n - 1$  et que le message est envoyé initialement par l'entité de calcul 0, il est possible d'utiliser la structure de tas binaire pour résoudre le problème de la diffusion :

- Les fils de l'entité de calcul d'identifiant  $i$  sont les entités de calcul d'identifiants  $2i + 1$  et  $2i + 2$  lorsqu'elles existent,
- Le père de l'entité de calcul d'identifiant  $i$  ( $> 0$ ) est l'entité de calcul d'identifiant  $\lfloor (i - 1)/2 \rfloor$ .

L'algorithme CAST peut alors être utilisé de la manière suivante, sans connaître la structure d'un arbre couvrant :

**Si** identifiant  $\neq 0$  **alors**,

Recevoir  $M$  de l'entité de calcul d'identifiant  $\lfloor (i - 1)/2 \rfloor$ ,

**Fin si**

**Si**  $2\text{identifiant} + 1 < n$  **alors**,

Envoyer  $M$  à l'entité de calcul d'identifiant  $2\text{identifiant} + 1$ ,

**Fin si**

**Si**  $2\text{identifiant} + 2 < n$  **alors**,

Envoyer  $M$  à l'entité de calcul d'identifiant  $2\text{identifiant} + 2$ .

**Fin si**

## III.2 Le calcul d'un arbre couvrant

Lorsque les hypothèses énoncées dans la section III.1.3 ne sont pas satisfaites, l'algorithme CAST nécessite de connaître un arbre couvrant. Un tel arbre peut être

obtenu à l'aide de l'algorithme FLOOD. En général, il est possible de contruire un arbre couvrant à partir de n'importe quel algorithme de diffusion.

**Proposition.** Tout algorithme de diffusion à partir d'une entité de calcul  $r$  permet de construire un arbre couvrant de racine  $r$ .

En effet, lorsqu'un algorithme de diffusion à partir de l'entité de calcul  $r$  s'exécute, il suffit de définir de père d'une entité de calcul  $v$  comme l'entité de calcul de laquelle  $v$  reçoit le message (diffusé à partir de  $r$ ) en premier. Il n'est ainsi pas possible de créer un cycle. D'autre part, l'arbre construit est bien couvrant puisque le message est transmis à toutes les entités de calcul.

**Remarque.** Afin que chaque entité de calcul dispose de la liste de ses fils, il faut que chaque entité de calcul ré-émette un message vers son père, et il faut mettre en place la réception de ce type de message.

Par conséquent :

**Proposition.** Il existe un algorithme distribué noté (*SPANTREE*) obtenu par la proposition précédente avec l'algorithme FLOOD qui permet de construire un arbre couvrant du graphe d'un système distribué de racine  $r$ , et dont la complexité est :

- i.  $\text{MESSAGE}(\text{SPANTREE}, G) = O(m)$ ,
- ii.  $\text{TEMPS}(\text{SPANTREE}, G) = O(\text{ecc}(r))$ .

### III.2.1 Détection de la terminaison

Sans plus de précision, l'algorithme précédent pour la construction d'un arbre couvrant ne peut pas être implémenté. En particulier, les entités de calcul ne savent jamais si la diffusion est terminée ou non. C'est un problème important en algorithmique distribuée.

Pour résoudre ce problème, il suffit de modifier l'algorithme de diffusion afin que chaque sommet reçoive un signal de fin. Ce message est envoyé conjointement avec l'accusé de réception qui indique à une entité de calcul qu'elle est le père (ou non) d'une autre entité de calcul. Chaque entité de calcul  $u$  dispose des variables suivantes :

- Une variable  $\text{PÈRE}(u)$  qui contient l'identifiant de son père,
- Une liste  $\text{FILS}(u)$  de ses fils,

- Une variable  $FIN(u)$  qui vaut 1 lorsque la diffusion est terminée et 0 sinon.

L'algorithme de diffusion et de construction d'un arbre couvrant avec détection de la terminaison peut s'écrire de la manière suivante (dans cet algorithme,  $u$  est l'identifiant de l'entité de calcul courante) :

**Poser**  $FILS(u) \leftarrow \emptyset$  et  $FIN(u) \leftarrow 0$ .

**Si**  $u = r$  **alors**,

Envoyer le message  $(M, donnée)$  à tous les voisins de  $u$ ,

Poser  $c \leftarrow \deg(u)$ . (★)

**Sinon**

Recevoir  $M$  de l'entité de calcul  $v$ ,

Envoyer le message  $(M, donnée)$  à tous les voisins de  $u$  différents de  $v$ ,

Poser  $PÈRE(u) \leftarrow v$ ,

Poser  $c \leftarrow \deg(u) - 1$ . (★★)

**Fin si**

**Tant que**  $c > 0$  **faire**

Recevoir  $M'$  de l'entité de calcul  $v$ ,

**Si**  $M'$  est de type *donnée* **alors**,

Envoyer le message *accusé de réception* à  $v$ . (★★★)

**Fin si**

**Si**  $M'$  est de type *père* **alors**,

Poser  $FILS(u) \leftarrow FILS(u) \cup \{v\}$ .

Poser  $c \leftarrow c - 1$ . (★★★★)

**Fin si**

**Si**  $M'$  est de type *accusé de réception* **alors**,

Poser  $c \leftarrow c - 1$ .

**Fin si**

**Fin tant que**

**Si**  $u \neq r$  **alors**,

Envoyer le message *père* à  $PÈRE(u)$ ,

Recevoir le message *fin* de  $PÈRE(u)$ .

**Fin si**

**Poser**  $FIN(u) \leftarrow 1$ ,

Envoyer *fin* à tous les  $v \in FILS(u)$ .

Cet algorithme sera noté FLOOD&ECHO( $r$ ).

**Remarques.** Dans l'algorithme FLOOD&ECHO( $r$ ), les messages de type *donnée*, *père*, et *accusé de réception* peuvent être discriminés en utilisant la balise de MPI. Il est important de noter que cet algorithme ne fonctionne que si deux messages envoyés d'une entité de calcul  $i$  à une entité de calcul  $j$  arrivent à  $j$  dans l'ordre dans lequel ils ont été envoyés. Cette hypothèse est fausse dans la librairie MPI. Pour contourner ce problème, il suffit d'attendre deux messages en retour pour chaque message envoyé : l'un de ces messages est un accusé de réception, et l'autre un message de type donnée. Un message de type père compte pour les deux à la fois. Ainsi, il s'agit de modifier l'algorithme de la façon suivante :

- remplacer  $(\star)$  par « Poser  $c \leftarrow 2 \deg(u)$  »,
- remplacer  $(\star\star)$  par « Poser  $c \leftarrow 2 \deg(u) - 2$  »,
- ajouter « Poser  $c \leftarrow c - 1$  » après  $(\star\star)$ ,
- remplacer  $(\star\star\star)$  par « Poser  $c \leftarrow c - 2$  ».

**Proposition.** En mode synchrone ou asynchrone,

- i. MESSAGE(FLOOD&ECHO( $r$ ),  $G$ ) =  $O(m)$ ,
- ii. TEMPS(FLOOD&ECHO( $r$ ),  $G$ ) =  $O(\text{ecc}(r))$ .

*Preuve.* Le message  $(M, \text{donnée})$  est envoyé autant de fois qu'il y a d'incidences sommêt-arêtes dans le graphe, c'est-à-dire  $2m$  fois. L'envoi des accusés de réception (comprenant les messages *père*) est fait le long de l'arbre, il faut donc  $n - 1$  messages. L'envoi des messages *fin* lui aussi le long de l'arbre nécessite  $n - 1$  messages. Donc :

$$\text{MESSAGE}(\text{FLOOD\&ECHO}(r), G) = 2m + 2n - 2.$$

Puisque le graphe est connexe,  $m \geq n - 1$ , et on obtient

$$\text{MESSAGE}(\text{FLOOD\&ECHO}(r), G) = O(m).$$

En outre la diffusion du message  $(M, \text{donnée})$  nécessite au plus un temps de  $\text{ecc}(r)$ . De même, l'envoi des accusés de réception et des messages *fin* nécessite au plus un temps de  $\text{ecc}(r)$  (il s'agit, en fait, de la hauteur de l'arbre, mais cette hauteur n'est pas connue, et elle est bornée par l'eccentricité de  $r$ ).  $\square$



### III.2.2 Le problème de la concentration

Le problème de la concentration est inverse au problème de la diffusion : chaque entité de calcul dispose d'une information  $M_u$ , et elle doit transmettre cette information à un sommet particulier  $r$ . Lorsqu'un arbre  $T$  de racine  $r$  est disponible, le problème de la concentration peut être résolu par l'algorithme suivant, noté  $\text{CONC}_T(r)$  :

```
Poser  $c \leftarrow |\text{FILS}(u)|$ ,  
Si  $c = 0$  et  $u \neq r$  alors,  
    Envoyer le message  $M_u$  à  $\text{PÈRE}(u)$ ,  
Sinon  
    Poser  $M \leftarrow M_u$   
    Tant que  $c > 0$ ,  
        Recevoir message  $M'$  de  $v$ ,  
        Poser  $M \leftarrow M \cup M'$  et  $c \leftarrow c - 1$ ,  
    Fin tant que  
    Si  $u \neq r$  alors,  
        Envoyer le message  $M$  à  $\text{PÈRE}(u)$ ,  
    Fin si  
Fin si
```

**Remarque.** Lors de l'exécution de cet algorithme, chaque entité de calcul concatène son message avec les messages qu'elle reçoit de ses fils avant d'envoyer un unique message à son père.

### III.2.3 Le problème de la réduction

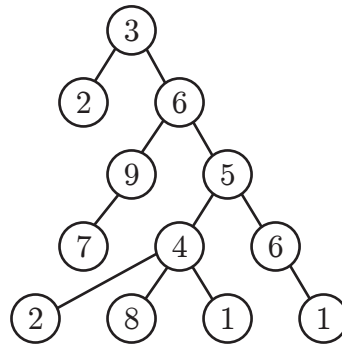
Le problème de la réduction est voisin de celui de la concentration. Ici, les messages sont des nombres et, à la réception d'un message à partir d'un de ses fils, chaque noeud effectue une certaine opération  $op$  ( $=\max, \min, +, \dots$ ) avec son propre message. Lorsqu'un arbre  $T$  de racine  $r$  est disponible, le problème de la réduction vers  $r$  peut être résolu par l'algorithme suivant, noté  $\text{REDUC}_T(r)$  :

```
Poser  $c \leftarrow |\text{FILS}(u)|$ ,  
Si  $c = 0$  et  $u \neq r$  alors,  
    Envoyer le message  $M_u$  à  $\text{PÈRE}(u)$ ,  
Sinon
```

**Poser**  $M \leftarrow M_u$   
**Tant que**  $c > 0$ ,  
     Recevoir message  $M'$  de  $v$ ,  
     **Poser**  $M \leftarrow op(M, M')$  et  $c \leftarrow c - 1$ ,  
**Fin tant que**  
**Si**  $u \neq r$  **alors**,  
     Envoyer le message  $M$  à  $PÈRE(u)$ ,  
**Fin si**  
**Fin si**

**Remarque.** L'opération  $op$  doit être associative :  $op(op(a, b), c) = op(a, op(b, c))$ . Sinon le résultat final dépend de l'arbre  $T$  !

**Exemple.** Effectuer l'algorithme de réduction avec l'arbre suivant et l'addition ( $op(a, b) = a + b$ ). La valeur indiquée sur le noeud  $u$  est  $M_u$  :



### III.2.4 Arbre de parcours en largeur

En mode synchrone, et dans le cas où  $G$  est complet, l'exécution de l'algorithme de construction d'un arbre couvrant basé sur l'algorithme d'inondation résulte en un arbre de hauteur 1 (la racine  $r$  est le père de tous les autres sommets). En effet, pendant la première unité de temps, le message  $M$  est envoyé par  $r$  à toutes les entités de calcul, qui choisissent donc  $r$  comme père. Ceci n'est pas forcément le cas en mode asynchrone. En particulier l'arbre obtenu n'est pas forcément un arbre de parcours en largeur :

**Définition.** Un arbre de parcours en largeur dans  $G$  de racine  $r$  est un arbre recouvrant  $T$  de  $G$  tel que pour tout sommet  $v$  de  $G$ ,

$$d_T(r, v) = d_G(r, v).$$

Ce type d'arbre *ne rallonge pas* la distance d'un sommet à la racine. Pour obtenir un tel arbre, il suffit d'appliquer l'algorithme de Bellman-Ford. Pour cet algorithme, chaque entité de calcul  $u$  dispose des variables suivantes :

- Une variable  $PÈRE(u)$  qui contient l'identifiant de son père,
- Une variable  $L(u)$  qui est la distance de  $u$  à  $r$ . Cette distance est mise à jour au fur et à mesure de l'exécution de l'algorithme.

L'algorithme peut être décrit de la manière suivante :

**Poser**  $FILS(u) \leftarrow \emptyset$  et  $FIN(u) \leftarrow 0$ .

**Si**  $u = r$  **alors**,

Envoyer le message  $(0, \textit{donnée})$  à tous les voisins de  $u$ ,

Poser  $c \leftarrow 2 \deg(u)$ .

**Sinon**

Recevoir  $d$  de l'entité de calcul  $v$ ,

Poser  $L(u) \leftarrow d + 1$ ,  $PÈRE(u) \leftarrow v$  et  $c \leftarrow 2 \deg(u) - 2$ ,

Envoyer le message  $(d + 1, \textit{donnée})$  à tous les voisins de  $u$  sauf  $v$ .

**Fin si**

**Tant que**  $c > 0$  **faire**

Recevoir  $M'$  de l'entité de calcul  $v$ ,

**Si**  $M'$  est de type  $(d, \textit{donnée})$  **alors**,

**Si**  $L(u) > d + 1$  **alors**,

Envoyer le message  $(d + 1, \textit{donnée})$  à  $PÈRE(u)$ .

Envoyer le message *accusé de réception* à  $PÈRE(u)$ .

Poser  $L(u) \leftarrow d + 1$  et  $PÈRE(u) \leftarrow v$ .

**Sinon**

Envoyer le message *accusé de réception* à  $v$ .

**Fin si**

Poser  $c \leftarrow c - 1$ .

**Fin si**

**Si**  $M'$  est de type *père* **alors**,

Poser  $FILS(u) \leftarrow FILS(u) \cup \{v\}$ .

Poser  $c \leftarrow c - 2$ .

**Fin si**

**Si**  $M'$  est de type *accusé de réception* **alors**,

Poser  $c \leftarrow c - 1$ .

**Fin si**

**Fin tant que**

**Si**  $u \neq r$  **alors**,

Envoyer le message *père* à  $PÈRE(u)$ ,

**Fin si**

### III.2.5 Algorithme de Dijkstra

L'algorithme de Dijkstra permet lui-aussi de construire un arbre de parcours en largeur. Il est plus compliqué que l'algorithme de la section précédente. Il se déroule en  $ecc_G(r)$  étapes. Chaque étape consiste à ajouter une *couche* à l'arbre partiellement construit :

**Poser**  $FILS(u) \leftarrow \emptyset$  et  $FIN(u) \leftarrow 0$ .

**Si**  $u = r$  **alors**,

**Poser**  $FEUILLE(u) \leftarrow 1$

**Sinon**

**Poser**  $FEUILLE(u) \leftarrow 0$

**Fin si**

**Tant que**  $FIN(u) == 0$  **faire**,

**Si**  $u = r$  **alors**,

**Si**  $FEUILLE(u) == 1$  **alors**,

Envoyer le message *couche* à tous les voisins de  $u$ ,

**Poser**  $c \leftarrow \deg(u)$ .

**Sinon**

Envoyer le message *pulse* à tous les fils de  $u$ ,

**Poser**  $c \leftarrow |FILS(u)|$ .

**Fin si**

**Poser**  $FIN(u) \leftarrow 1$

**Tant que**  $c > 0$  **faire**,

Recevoir  $M$  de l'entité de calcul  $v$ ,

**Si**  $M$  de type *couche* **alors**,

Envoyer le message *accusé de réception* à  $v$ ,

**Fin si**

**Si**  $M$  de type *père* **alors**,  
 Poser  $\text{FILS}(u) \leftarrow \text{FILS}(u) \cup \{v\}$ ,  $\text{FEUILLE}(u) \leftarrow 0$ ,  $c \leftarrow c - 1$   
 et  $\text{FIN}(u) \leftarrow 0$ .  
**Fin si**  
**Si**  $M$  de type *accusé de réception* **alors**,  
 Poser  $c \leftarrow c - 1$  et  $\text{FIN}(u) \leftarrow 0$ .  
**Fin si**  
**Si**  $M$  de type *fin* **alors**,  
 Poser  $c \leftarrow c - 1$ .  
**Fin si**  
**Fin tant que**  
**Sinon**  
 Recevoir  $M$  de l'entité de calcul  $v$ ,  
**Si**  $M$  de type *couche* **alors**,  
 Envoyer le message *père* à l'entité de calcul  $v$ ,  
**Poser**  $\text{FEUILLE}(u) \leftarrow 1$  et  $c \leftarrow 0$ .  
**Sinon** [ $M$  est nécessairement de type *pulse* et  $v = \text{PERE}(u)$ ]  
**Si**  $\text{FEUILLE}(u) == 1$  **alors**,  
 Envoyer le message *couche* à tous les voisins de  $u$  sauf  $v$ ,  
**Poser**  $c \leftarrow \deg(u) - 1$ .  
**Sinon**  
 Envoyer le message *pulse* à tous les fils de  $u$ ,  
**Poser**  $c \leftarrow |\text{FILS}(u)|$ .  
**Fin si**  
**Poser**  $\text{FIN}(u) \leftarrow 1$   
**Tant que**  $c > 0$  **faire**,  
 Recevoir  $M$  de l'entité de calcul  $w$ ,  
**Si**  $M$  de type *couche* **alors**,  
 Envoyer le message *accusé de réception* à  $w$ ,  
**Fin si**  
**Si**  $M$  de type *père* **alors**,  
 Poser  $\text{FILS}(u) \leftarrow \text{FILS}(u) \cup \{w\}$ ,  $\text{FEUILLE}(u) \leftarrow 0$ ,  $c \leftarrow c - 1$   
 et  $\text{FIN}(u) \leftarrow 0$ .  
**Fin si**

**Si**  $M$  de type *accusé de réception* **alors**,  
    Poser  $c \leftarrow c - 1$   
    **Si** FEUILLE( $u$ )  $\neq 1$  **alors**,  
        Poser FIN( $u$ )  $\leftarrow 0$ .  
    **Fin si**  
**Fin si**  
**Si**  $M$  de type *fin* **alors**,  
    Poser  $c \leftarrow c - 1$ .  
**Fin si**  
**Fin tant que**  
**Si** FIN( $u$ )  $== 1$  **alors**,  
    Envoyer le message *fin* à l'entité de calcul  $v$ ,  
**Sinon**  
    Envoyer le message *accusé de réception* à l'entité de calcul  $v$ ,  
**Fin si**  
**Fin si**  
**Fin si**  
**Fin tant que**

## IV Communications collectives avec MPI

Lors d'une communication collective, toutes les entités de calcul participent à la communication. La diffusion, la concentration et la réduction sont des exemples de communications collectives. Avec MPI, il s'agit de *toutes les entités de calcul d'un communicateur* (par exemple MPI\_COMM\_WORLD).

Il n'y a pas de synchronisation : un processus peut avoir terminé avant que d'autres processus n'aient commencé.

### IV.1 Diffusion

Pour diffuser un message à toutes les entités de calcul d'un communicateur, on utilise la commande MPI\_Bcast (Broadcast) :

MPI\_Bcast(buffer, k, MPI\_Type, source, MPI\_COMM\_WORLD);

Lors de cette commande, toutes les entités de calcul ont déclaré un tableau `buffer` (typiquement avant l'initialisation de MPI). Les  $k$  premières cases du tableau `buffer` de l'entité source sont recopiées dans les  $k$  premières cases du tableau `buffer` de toutes les autres entités.

### Exemple.

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"

int main()
{
    int size, rank, message;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank==0)
        message=42;

    MPI_Bcast(&message, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Finalize();
    return EXIT_SUCCESS;
}
```

## IV.2 Concentration

Pour concentrer des messages vers une entité de calcul  $r$ , on utilise la commande `MPI_Gather`. Toutes les entités de calcul  $e \neq r$  envoient un message de longueur  $k_e$ . L'entité de calcul  $r$  reçoit  $n - 1$  messages de longueur  $k_r$ , où  $n$  est le nombre total d'entités de calcul :

```
MPI_Gather(buffer_e, k_e, MPI_Type, buffer_r, k_r, MPI_Type,
          r, MPI_COMM_WORLD);
```

Lors de cette commande, le buffer `buffer_r` ne sert que sur l'entité de calcul *r*. toutes les entités de calcul ont déclaré un tableau `buffer` (typiquement avant l'initialisation de MPI). Les *k* premières cases du tableau `buffer` de l'entité source sont recopiées dans les *k* premières cases du tableau `buffer` de toutes les autres entités.

### Exemple.

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"

int main()
{
    int size, rank;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int message, *buffer_r;

    if(rank!=0)
        message=rank;

    buffer_r=(int *)malloc((size-1)*sizeof(int));

    MPI_Gather(&message, 1, MPI_INT, buffer_r, 1, MPI_INT,
              0, MPI_COMM_WORLD);

    MPI_Finalize();
    return EXIT_SUCCESS;
}
```



## IV.3 Réduction

Pour effectuer le problème de la réduction vers une entité de calcul  $r$ , on utilise la commande `MPI_Reduce`. Toutes les entités de calcul  $e \neq r$  envoient un message de longueur  $k_e$ . L'entité de calcul  $r$  reçoit  $n - 1$  messages de longueur  $k_r$ , où  $n$  est le nombre total d'entités de calcul :

```
MPI_Reduce(buffer_e, buffer_r, k, MPI_Type, MPI_Op,  
           r, MPI_COMM_WORLD);
```

Les buffers `buffer_e` et `buffer_r`, respectivement utilisés pour l'envoi et la réception contiennent  $k$  valeurs de type `MPI_Type`. Si  $k > 1$ , l'opération est effectuée termes à termes entre le buffer reçu et le buffer courant de l'entité de calcul. L'opération `MPI_Op` peut être :

- `MPI_MAX` : maximum,
- `MPI_MIN` : minimum,
- `MPI_SUM` : somme,
- `MPI_PROD` : produit,
- `MPI_LAND` : et logique,
- `MPI_LOR` : ou logique...

**Exemple.**

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"

int main()
{
    int size, rank;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int rec, env;
```

```

if(rank!=0)
    env=rank;

MPI_Reduce(&env, &rec, 1, MPI_INT, MPI_SUM,
                                                  0, MPI_COMM_WORLD);

MPI_Finalize();
return EXIT_SUCCESS;
}

```

## IV.4 Barrière de synchronisation

Une barrière de synchronisation permet à chaque processus d'attendre que tous les autres processus aient terminé un calcul :

```
MPI_Barrier(MPI_COMM_WORLD);
```

En pratique, la commande `MPI_Barrier` ne se termine pour une entité de calcul donnée que lorsque toutes les autres entités ont commencé à l'exécuter.

## V Communicateurs

Les communications collectives se font pour toutes les entités de calcul d'un communicateur. Afin de pouvoir les utiliser de manière flexible, MPI permet de définir de nouveaux communicateurs à partir de `MPI_COMM_WORLD` qui ne contiennent qu'une partie des entités de calcul.

### V.1 Partition d'un communicateur

Il est possible de partitionner les entités de calcul d'un communicateur `comm` en plusieurs communicateurs plus petits avec la fonction `MPI_Comm_split` :

```
MPI_Comm_split(comm, couleur, nouvel_id, &nouveau_comm);
```

Toutes les entités de calcul du communicateur `comm` doivent appeler cette fonction. Le communicateur `nouveau_comm` doit être déclaré sur chaque entité de calcul, mais sa valeur après l'appel de `MPI_Comm_split` sera différente d'une entité de calcul à l'autre. Toutes les entités de calcul qui ont la même couleur (un entier) seront regroupées dans le même nouveau communicateur.

L'entier `nouvel_id` donne l'identifiant qu'aura chaque entité de calcul dans son nouveau communicateur. En particulier, chaque nouveau communicateur doit avoir une entité d'identifiant 0.

Il est possible d'imposer qu'une entité de calcul n'appartienne pas à l'un des nouveaux communicateurs en précisant `MPI_UNDEFINED` à la place de la couleur. Dans ce cas, la valeur de `nouveau_comm` sera `MPI_COMM_NULL` après l'appel de la fonction.

**Exemple (partition pair/impair).**

```
MPI_Comm_split(comm, id%2, id-id%2, &nouveau_comm);
```

## V.2 Topologie cartésienne

Permet d'organiser les entités de calcul selon différentes dimensions. Par exemple, en dimension 2, les entités de calcul seront dans une ligne et dans une colonne selon une grille carrée. Par exemple avec 8 communicateurs :

|         |         |         |         |
|---------|---------|---------|---------|
| 4 (1,0) | 5 (1,1) | 6 (1,2) | 7 (1,3) |
| 0 (0,0) | 1 (0,1) | 2 (0,2) | 3 (0,3) |

Ici les paires entre parenthèses sont les coordonnées cartésiennes d'une entité de calcul, et le nombre à gauche est son identifiant dans le communicateur. Il est bien-sûr possible de définir ce type d'organisation des entités de calcul, appelée *topologie cartésienne*, en dimension quelconque. Dans une telle topologie, chaque entité de calcul communique avec ses voisins immédiats selon chaque dimension. Pour créer un tel communicateur (appelons-le `nouveau_comm`) à partir du communicateur `comm`, on utilise la fonction `MPI_Cart_create` :

```
MPI_Cart_create(comm, d, tailles, periodes, r, &nouveau_comm);
```

Ici, l'entier  $d = 1, 2, \dots$  précise le nombre de dimensions; `tailles` et `periodes` sont deux tableaux de  $d$  entiers (`tailles[0], ..., tailles[d-1]` et `periodes[0], ..., periodes[d-1]`). Le premier tableau précise le nombre d'entités de calcul dans chaque dimension. La valeur de `periodes[i]` est soit 0 soit 1 selon que la dimension  $i+1$  est périodique ou non. Enfin `r` précise si les entités de calcul peuvent être réordonnées dans le nouveau communicateur (dans ce cas, les identifiants dans le nouveau communicateur seront modifiés).

### V.2.1 calcul du tableau des tailles

Le tableau des tailles peut être rempli par la fonction `MPI_Dims_create` :

```
MPI_Dims_create(nb_processus, d, tailles);
```

Avant l'appel de cette fonction, les composantes du tableau `tailles` doit être mises à 0. Si une des composantes n'est pas nulle, la fonction supposera que le nombre précisé est la taille voulue de la dimension correspondante.

Attention, il faut que `nb_processus` soit divisible par  $d$ , et que le pré-remplissage du tableau `tailles` n'empêche pas la partition du communicateur :

| tailles avant | appel                                         | tailles après |
|---------------|-----------------------------------------------|---------------|
| [0,0]         | <code>MPI_Dims_create(9, 2, tailles);</code>  | [3,3]         |
| [0,0]         | <code>MPI_Dims_create(13, 2, tailles);</code> | [13,1]        |
| [0,2,0]       | <code>MPI_Dims_create(12, 3, tailles);</code> | [3,2,2]       |
| [0,0]         | <code>MPI_Dims_create(7, 2, tailles);</code>  | erreur !      |
| [0,5,0]       | <code>MPI_Dims_create(12, 3, tailles);</code> | erreur !      |

### V.2.2 Recherche de coordonnées

Il est possible de trouver les coordonnées cartésiennes d'une entité de calcul à partir de son identifiant *dans un communicateur comm avec une topologie cartésienne* en utilisant la fonction `MPI_Cart_coords` :

```
MPI_Cart_coords(comm, identifiant, d, coord);
```

Ici `coord` est un tableau de  $d$  entiers qui sera rempli par la fonction.

**Exemple.** Reprenons le communicateur `comm` décrit plus haut :

|         |         |         |         |
|---------|---------|---------|---------|
| 4 (1,0) | 5 (1,1) | 6 (1,2) | 7 (1,3) |
| 0 (0,0) | 1 (0,1) | 2 (0,2) | 3 (0,3) |

L'appel `MPI_Cart_coords(comm, 6, 2, coord);` remplira le tableau `coord` avec les entiers 1 et 2 : `coord[0]=1` et `coord[1]=2`.

### V.2.3 Recherche de voisins

Il est possible de trouver les identifiants des entités de calcul voisines selon la dimension  $k$  *dans un communicateur comm avec une topologie cartésienne* en uti-

lisant la fonction `MPI_Cart_shift` :

```
MPI_Cart_shift(comm, k, s, &prec, &suiv);
```

Ici `s` est un entier qui vaut 1 si on cherche les voisins immédiat, 2 si on cherche les entités de calcul à distance 2 dans la dimension `k`, etc... Les entiers `prec` et `suiv` sont remplis par la fonction.

Lorsqu'on se décale de `s` pas dans la dimension `k` à partir du processus courant, il est possible qu'on sorte du graphe. Dans ce cas la valeur retournée de `prec` ou de `suiv` est `MPI_PROC_NULL`. Si cette dimension est périodique, ce problème ne peut pas se produire !

**Exemple.** Reprenons le communicateur `comm` décrit plus haut :

|         |         |         |         |
|---------|---------|---------|---------|
| 4 (1,0) | 5 (1,1) | 6 (1,2) | 7 (1,3) |
| 0 (0,0) | 1 (0,1) | 2 (0,2) | 3 (0,3) |

L'appel `MPI_Cart_shift(comm, 0, 6, &prec, &suiv);` sur l'entité de calcul 6 retournera `prec=5` et `suiv=7`. Si la dimension verticale est non-périodique, l'appel `MPI_Cart_shift(comm, 1, 6, &prec, &suiv);` sur l'entité de calcul 6 retournera `prec=2` et `suiv=MPI_PROC_NULL`. Si cette dimension est périodique, cet appel retournera `prec=2` et `suiv=2`.

#### V.2.4 Sous-communicateurs dimensionnels

Si `comm` est un communicateur avec une topologie cartésienne de dimension `d`, il est possible de définir un nouveau communicateur `nouveau_comm` qui partitionne `comm` selon certaines dimensions. Il suffit pour cela d'utiliser la commande `MPI_Cart_sub` :

```
MPI_Cart_sub(comm, d_restantes, &nouveau_comm);
```

Ici, `d_restantes` est un tableau de `d` entiers dont la `i`-ème composante est 0 (si la dimension `i` n'est pas dans le nouveau communicateur) ou 1 (si la dimension `i` est dans le nouveau communicateur). Dans le premier cas, deux entités de calcul de composantes différentes selon la dimension `i` seront dans deux nouveaux communicateurs différents.

**Exemple.** Reprenons le communicateur `comm` décrit plus haut :

|         |         |         |         |
|---------|---------|---------|---------|
| 4 (1,0) | 5 (1,1) | 6 (1,2) | 7 (1,3) |
| 0 (0,0) | 1 (0,1) | 2 (0,2) | 3 (0,3) |

Considérons l'appel

```
MPI_Cart_sub(comm, d_restantes, &nouveau_comm);
```

Si `d_restantes[0]=1` et `d_restantes[1]=0`, deux nouveaux communicateurs seront créés, correspondant à chaque ligne du tableau (tous les deux seront créés dans `nouveau_comm`; chaque entité de calcul n'aura accès qu'au nouveau communicateur auquel elle appartient). Le premier contient les entités 0, 1, 2 et 3, et l'autre communicateur contient les entités 4, 5, 6 et 7.

Si `d_restantes[0]=0` et `d_restantes[1]=1`, quatre nouveaux communicateurs seront créés, correspondant aux colonnes. Le premier contient les entités 0 et 4, le deuxième les entités 1 et 5, etc...

## Références

- [1] Robert G. Gallager, Pierre A. Humblet, P. M. Spira, *A distributed algorithm for minimum-weight spanning trees*, ACM Transactions on Programming Languages and Systems **5**(1), 66–77 (1983)
- [2] Cyril Gavoille, *Algorithmes distribués*, Cours de Master 1 et 2, Université de Bordeaux (2015)